
Fitt4Sences

Resumen ejecutivo del Trabajo de Fin de Grado

Aplicacion Android nativa para el seguimiento integral de bienestar personal Kotlin - Jetpack
Compose - MVVM - Supabase - PostgreSQL

Documento preparado como resumen profesional para GitHub, portfolio y defensa tecnica.

1. Resumen ejecutivo

Fitt4Sences es una aplicacion Android nativa orientada al seguimiento integral del bienestar personal. El proyecto integra en una sola herramienta cuatro pilares principales: nutricion, ejercicio fisico, hidratacion y metricas corporales. Su valor tecnico no esta solo en registrar datos, sino en relacionarlos para ofrecer una vision diaria coherente del estado del usuario.

Idea clave: la app no delega los calculos criticos en la interfaz movil. La base de datos PostgreSQL actua como fuente de verdad y calcula TMB, IMC, calorias quemadas, hidratacion y balance energetico.

65	9	26	13	48
archivos Kotlin	pantallas	componentes UI	tablas BD	politicas RLS

2. Problema que resuelve

El proyecto nace de una necesidad clara: evitar la fragmentacion habitual de las apps de salud. Muchas soluciones se centran en un unico area y obligan al usuario a combinar manualmente alimentacion, ejercicio, hidratacion y metricas fisicas.

Problema detectado	Impacto en el usuario	Respuesta de Fitt4Sences
Datos aislados	El usuario no obtiene una vision completa de su estado diario.	Unifica nutricion, ejercicio, hidratacion y metricas en un dashboard central.
Calculos inconsistentes	Si los calculos viven solo en el cliente, pueden variar entre dispositivos o sesiones.	Delegacion de calculos criticos en PostgreSQL mediante funciones, triggers y columnas generadas.
Falta de privacidad real	Los datos de salud requieren aislamiento por usuario.	Row Level Security en Supabase/PostgreSQL para que cada usuario acceda solo a sus propios datos.

3. Objetivos principales

- Diseñar y desarrollar una app Android funcional para controlar bienestar personal desde una unica plataforma.
- Implementar autentificacion segura con registro, confirmacion por correo, recuperacion de contraseña y sesion persistente.
- Crear modulos operativos de nutricion, ejercicio, hidratacion y metricas fisicas.
- Aplicar arquitectura MVVM con repositorios y estado reactivo mediante StateFlow.
- Integrar Supabase como backend y PostgreSQL como motor activo de logica de negocio.
- Garantizar consistencia, seguridad y trazabilidad mediante RLS, restricciones, funciones y triggers.

4. Arquitectura tecnica

La solucion se organiza en tres capas principales, con separacion clara de responsabilidades. La interfaz captura eventos y renderiza estado; los ViewModels coordinan el flujo; los repositorios

encapsulan el acceso al backend; y PostgreSQL ejecuta la logica critica.

Capa	Responsabilidad	Tecnologias / elementos
Presentacion	Renderizar la UI, capturar acciones del usuario y observar estado reactivo.	Jetpack Compose, Material Design 3, Navigation Compose, StateFlow.
Datos	Encapsular consultas, operaciones CRUD e invocaciones RPC.	Repositorios Kotlin, Supabase Kotlin SDK, PostgREST, Ktor, kotlinx.serialization.
Backend	Persistencia, calculos, seguridad, integridad y reglas de negocio.	Supabase Auth, PostgreSQL 17, funciones SQL, triggers, RLS, enums, CHECK constraints.

Flujo principal: View/Composable -> ViewModel -> Repository -> Supabase/PostgREST/RPC -> PostgreSQL -> Repository -> ViewModel -> StateFlow -> Compose.

5. Modulos funcionales

Modulo	Estado	Funcionalidades clave
Autenticacion	Completo	Login, registro, confirmacion por correo, recuperacion de contraseña mediante deeplink y sesion persistente.
Dashboard	Completo	Resumen central con datos de usuario, IMC, TMB, metricas fisicas y balance energetico diario.
Nutricion	Completo	Registro y eliminacion de comidas por tipo, suma de calorías y recalculo automatico del balance diario.
Ejercicio	Completo	Habitos por defecto, sesiones con duracion, acumulacion diaria, MET, calorías quemadas, logros y graficos semanales.
Hidratacion	Completo	Registro de vasos, objetivo diario, ml por vaso configurables y evolucion semanal.
Metricas fisicas	Completo	Peso, altura, edad, sexo, IMC automatico y TMB calculada desde base de datos.
Estadisticas	Parcial	Centrada actualmente en hidratacion y ejercicio; preparada para evolucion analitica global.

6. Stack tecnologico

Area	Tecnologia	Uso en el proyecto
Lenguaje	Kotlin	Lenguaje principal de la aplicacion Android.
UI	Jetpack Compose + Material 3	Interfaces declarativas, modernas y reutilizables.
Arquitectura	MVVM + StateFlow	Separacion de responsabilidades y estado reactivo.
Backend	Supabase	Autenticacion, PostgREST, RPC y gestion en la nube.

Area	Tecnologia	Uso en el proyecto
Base de datos	PostgreSQL 17	Persistencia relacional y logica de negocio en servidor.
HTTP / API	Ktor + Supabase SDK	Comunicacion asincrona con el backend.
Graficos	Vico Charts	Visualizacion de actividad e hidratacion semanal.
Build	Gradle Kotlin DSL	Configuracion del proyecto, dependencias y BuildConfig.

7. Base de datos y logica de negocio

La base de datos es uno de los elementos mas importantes del proyecto. No se limita a guardar registros: calcula, valida, relaciona y protege datos.

Elemento	Cantidad / ejemplo	Funcion
Tablas	13 tablas	Usuarios, metricas fisicas, nutricion diaria, comidas, habitos, sesiones, hidratacion, configuracion de ejercicio y tablas futuras.
Funciones SQL / RPC	16 funciones	Calculo de TMB, upsert de metricas, recalcule de balance, registro de sesiones e hidratacion.
Triggers	16 triggers	Calorias quemadas, timestamps automaticos y asignacion de usuario autenticado.
Politiclas RLS	48 politicas	Aislamiento de datos por usuario autenticado.
Restricciones	12 CHECK + FK + enums	Evitan datos invalidos como calorias negativas, MET nulo o vasos fuera de rango.

8. Seguridad

- Autenticacion con Supabase Auth y tokens JWT.
- Row Level Security en las tablas operativas para aislar datos por usuario.
- Credenciales gestionadas mediante local.properties y BuildConfig, evitando valores literales en el codigo fuente.
- Funciones SECURITY DEFINER para operaciones que necesitan consultar varias tablas sin romper el modelo RLS.
- Validacion doble: en cliente mediante ViewModel y en servidor mediante CHECK, FK, ENUM y politicas.

9. Pruebas y validacion

La validacion se realizo principalmente mediante pruebas funcionales manuales y consultas directas en Supabase. Aunque existen dependencias preparadas para JUnit, Espresso y Compose UI Test, las pruebas automatizadas quedan como mejora futura.

Area probada	Validacion realizada	Resultado esperado
Autenticacion	Registro, confirmacion, login, recovery y deeplinks.	Acceso seguro y navegacion correcta.

Area probada	Validacion realizada	Resultado esperado
Nutricion	Insercion/eliminacion de comidas y recalcu de balance.	Balance diario actualizado tras cada operacion.
Ejercicio	Registro de sesiones, acumulacion y calculo MET.	Calorias quemadas y estadisticas actualizadas.
Hidratacion	Registro de vasos, ajustes y evolucion semanal.	Progreso diario y grafico semanal correctos.
Seguridad	Intento de acceso a datos de otros usuarios.	PostgreSQL devuelve conjunto vacio por RLS.

10. Resultados finales

El resultado es un MVP funcional instalable en Android API 24 o superior, con modulos principales operativos y una base preparada para evolucionar a producto mas completo.

Indicador	Resultado
Producto	MVP funcional con autentificacion, dashboard, nutricion, ejercicio, hidratacion y metricas fisicas.
Codigo	65 archivos Kotlin, 9 pantallas, 26 componentes reutilizables, 9 ViewModels y 9 repositorios.
Backend	PostgreSQL 17 en Supabase con 13 tablas, 16 funciones, 16 triggers y 48 politicas RLS.
Competencias demostradas	Desarrollo Android, UI declarativa, arquitectura MVVM, acceso a datos, seguridad, SQL, documentacion y resolucion de problemas.

11. Limitaciones reconocidas

- La aplicacion es exclusivamente Android; no existe cliente iOS ni web en el MVP.
- El registro calorico es manual y todavia no usa el catalogo de alimentos preparado en la base de datos.
- No incluye modo offline ni sincronizacion diferida.
- No incorpora notificaciones push para recordatorios.
- Las pruebas son manuales; faltan tests unitarios, de integracion y de UI automatizados.
- La disponibilidad depende del servicio Supabase en la nube.

12. Mejoras futuras

- Conectar el catalogo de alimentos y la tabla comidas_alimentos al flujo de nutricion.
- Integrar chat con IA y recomendaciones personalizadas usando las tablas ya preparadas.
- Crear estadisticas globales semanales que consoliden todos los modulos.
- Añadir notificaciones push para hidratacion, comidas y ejercicio.
- Implementar tests automatizados para ViewModels, repositorios y pantallas Compose.
- Añadir modo offline con Room o DataStore y sincronizacion posterior con Supabase.
- Preparar clientes futuros web o iOS consumiendo el mismo backend PostgreSQL.

13. Uso recomendado en GitHub

Para GitHub, este resumen debe ir en la carpeta docs junto a la memoria completa. El README principal debe ser mas breve y orientado a portfolio: descripcion del producto, stack, capturas, arquitectura, funcionalidades y enlace a la documentacion.

Archivo recomendado	Uso
README.md	Presentacion rapida del proyecto para recruiters, empresas y evaluadores.
README_ES.md	Version en español del README si se desea mantener repositorio bilingue.
docs/TFG_RESUMEN_ES.pdf	Resumen ejecutivo en español para consulta rapida.
docs/TFG_SUMMARY_EN.pdf	Copia equivalente en ingles para portfolio internacional.
docs/TFG_Fitt4Sences.pdf	Memoria completa, solo si se reviso previamente que no contiene datos sensibles.

14. Conclusion

Fitt4Sences es un TFG solido porque combina una aplicacion Android moderna con un backend relacional real y seguro. Lo mas destacable es la decision de usar PostgreSQL como motor activo de logica de negocio, algo que eleva el proyecto por encima de una app de registro simple. Como pieza de portfolio, demuestra criterio arquitectonico, seguridad, integracion backend y capacidad para documentar un sistema completo.

Fuente utilizada: memoria del TFG Fitt4Sences proporcionada por el autor. Este documento resume los puntos esenciales y no sustituye a la memoria completa.